

Java script

JAVASCRIPT C'EST QUOI

- **JavaScript** est un langage de programmation coté client.
- C'est un langage qui va nous permettre de créer de l'**interaction** entre l'utilisateur et le client.
- Javascript est un langage de scripts qui s'incorpore au langage Html de présentation des pages Web.
- Les scripts vont permettre de rendre une page Web plus dynamique :
 - en animant certaines zones de la page,
 - ou en faisant réagir certains éléments de la page en fonction de certains événements provoqués par l'utilisateur.

Rôle de JavaScript dans le web

- HTML: Structure de la page
- CSS: Mise en forme
- JavaScript: Interaction & logique

Exemple :

- HTML : bouton
- CSS : couleur du bouton
- JS : action quand on clique

Où placer le JavaScript ?

Le JavaScript peut être placé à trois endroits différents :

- Dans l'élément **head** d'une page HTML.
- Dans l'élément **body** d'une page HTML.
- Dans un fichier portant l'extension **.js** séparé.

Dans l'élément head

- Dans le cas de l'insertion du code JavaScript à l'intérieur du head, il faut placer le JavaScript à l'intérieur d'une balise **script**.
- On place généralement l'élément script à la fin de notre élément head.

<head>

[...]

<script> alert("Au secours !!"); **</script>**

</head>

Dans l'élément body

- Dans le cas de l'insertion du code JavaScript à l'intérieur du body, il faut aussi placer le JavaScript à l'intérieur d'une balise script.

<script> alert("Au secours !!"); **</script>**

Dans un fichier JavaScript séparé

- Un fichier JavaScript a pour extension .js.
- est la méthode recommandée car elle permet une meilleure maintenabilité
- lier nos fichiers HTML et JavaScript en utilisant à nouveau un élément script et son attribut src.
- Si l'on souhaite lier un fichier JavaScript **script.js** à un fichier html placé dans le même dossier nous écrirons à la fin du head :
- `<script src="script.js"></script>` Dans le fichier **script.js**, on pourra écrire par exemple cette ligne :
`alert("JS...easy !");`

Afficher un message

- Affiche une fenêtre popup:

```
alert("Bienvenue en JavaScript");
```

- Afficher dans la console

```
console.log("Hello JavaScript");
```

Indenter

Indenter

- Une bonne pratique est d'effectuer un retrait vers la droite équivalent à une tabulation à chaque fois qu'on écrit une nouvelle ligne de code à l'intérieur d'une instruction JavaScript.

```
if (age >= 18) {  
    console.log("Accès autorisé");  
  
    if (age >= 21) {  
        console.log("Accès complet");  
    }  
}
```

Commenter

//: Commentaire sur une ligne

/* */ :Commentaire sur plusieurs lignes

Variables

- une variable JavaScript est un conteneur servant à stocker temporairement une information.
- Par convention, les noms des variables JavaScript suivent les règles du **camelCase** : tous les mots sont collés avec une majuscule en début de chaque mot, hormis le premier.
- Le nom des variables est **sensible à la casse**, c'est à dire qu'une variable portant le nom a n'est pas la même que celle qui porte le nom A.
- monNomDeVariable respecte le camelCase.
- Pour observer nos variables nous les afficherons dans la console de votre navigateur.
- Un bloc en JavaScript est délimité par des accolades { }.

Déclaration des variables

- Il faudra commencer par **déclarer** (=créer) les variables avant de pouvoir y affecter une valeur grâce à l'opérateur d'affectation =.
- Il y a trois manières de déclarer une variable en JavaScript : **let**, **const** et **var**.

L'instruction let

- L'instruction let permet de déclarer une variable de manière globale ou locale de manière simple.

```
<script> let maVar = false; console.log(maVar); </script>
```

- Il est possible de déclarer une variable puis plus tard de réaliser une affectation.

```
<script>
  let maVar2;
  console.log(maVar2); /* undefined */
  maVar2 = 3; console.log(maVar2);
</script>
```

L'instruction let

- Comme c'est une variable, on peut modifier sa valeur après sa déclaration :

```
<script> let maVar=true; console.log(maVar); maVar=maVar+maVar;
console.log(maVar); </script> /*true + true → 1 + 1 → 2true + true → 1 + 1 → 2/*
```

L'instruction let (Déclaration variable globale)

- **globale** si on déclare la variable au début d'élément script

```
<script>  
let maVar=1; // globale  
console.log(maVar);  
if (true) { maVar+=2; //locale  
console.log(maVar); }  
maVar+=1; console.log(maVar);  
</script>
```

L'instruction let (Déclaration variable locale)

- **locale** si on déclare la variable à l'intérieur bloc courant. C'est à dire **entre deux accolades à l'intérieur d'une fonction**, d'un if, d'un for

```
<script> if (true) { let maVara=2;  
console.log(maVara); } maVar+=1;  
console.log(maVara); </script>
```

L'instruction const

- L'instruction const permet de déclarer une variable globale ou locale constante et donc elle ne pourra pas être modifiée.
- `<script> const maVar4; </script>`
- À l'inverse de l'instruction let, l'instruction const a besoin d'être déclarée avec une valeur initiale.
- Écrire le code suivant :
- `<script> const maVar4; </script>` (problème initialisation)

L'instruction var

- L'instruction var est la variable qui au début du JavaScript permettait de déclarer des variables.
- Elle est à éviter depuis 2015.
- La variable var est **globale**
- Utilisez à la place les instructions let et const.
- **L'instruction var permet de :**
 - Déclarer une variable et lui assigner une valeur immédiatement :
`var x = 12;`
 - Déclarer une variable sans lui assigner de valeur et lui assigner une valeur plus tard :
`var temps; temps = 2;`
 - Déclarer plusieurs variables en même temps :
`var x = 12, prénom = "jean", nom = "neymar";`

Types de données

- **Number** : nombres entiers ou flottants → let age = 30;
- **String** : chaînes de caractères → let nom = « Mohamed »;
- **Boolean** : vrai/faux → let actif = true;
- **Null** : valeur vide intentionnelle → let vide = null;
- **Undefined** : variable déclarée sans valeur → let test;

Opérateurs arithmétiques

- + addition
- - soustraction
- * multiplication
- / division
- % modulo
- ** puissance

Opérateurs logiques

- && ET logique
- || OU logique
- ! NON logique

```
true && false; // false  
true || false; // true  
!true;        // false
```

Comparaison (== vs ===)

- == : compare les valeurs (conversion implicite).
- === : compare valeur et type (strict).

```
5 == "5"; // true  
5 === "5"; // false
```

Les structures conditionnelles

- Elles permettent au programme de **prendre des décisions**.
-  *Si une condition est vraie → ALORS exécuter un code*
- Syntaxe générale:

```
if (condition) {  
    // code exécuté si condition vraie  
} else {  
    // code exécuté sinon  
}
```

Exemple:

```
let age = 18;  
if (age >= 18) { console.log("Majeur"); } else  
{ console.log("Mineur"); }
```

Conditions (if, else, else if)

- `if (condition) { ... } else if (autreCondition) { ... } else { ... }`

```
let note = 12;  
  
if (note >= 16) {  
    console.log("Très bien");  
} else if (note >= 10) {  
    console.log("Admis");  
} else {  
    console.log("Ajourné");  
}
```

Conditions combinées

```
let age = 25;
let permis = true;

if (age >= 18 && permis === true) {
  console.log("Peut conduire");
}
```

switch

- **Quand utiliser switch ?**

Quand on teste **une seule variable** avec **plusieurs valeurs possibles**.

Syntaxes:

- **switch** (expression) { case valeur1: instructions; **break**; default: instructions; }

```
let jour = 3;
switch (jour) {
  case 1:
    console.log("Lundi");
    break;
  case 2:
    console.log("Mardi");
    break;
  case 3:
    console.log("Mercredi");
    break;
  default:
    console.log("Jour inconnu");
}
```

Break: est obligatoire pour éviter l'exécution des autres cas.

Les Boucles

Boucle for:

- `for (let i = 0; i < 5; i++) { console.log(i); }`

Boucle while:

- `let i = 0; while (i < 5) { console.log(i); i++; }`

(Le code peut s'exécuter 0 une fois.)

Boucle do...while:

- `let i = 0; do { console.log(i); i++; } while (i < 5);`

(Le code s'exécute au moins une fois.)

break et continue

- **break** : sort de la boucle.
- **continue** : passe à l'itération suivante.

Les fonctions

- **Problème sans fonction**
- `console.log("Bonjour Ali");`
`console.log("Bonjour Sami");`
`console.log("Bonjour Sara");`
- ✗ Répétition du code
- ✗ Difficile à maintenir

Solution avec fonction
`function direBonjour(nom) {`
 `console.log("Bonjour " +`
 `nom); }`

Appel de la fonction:
`direBonjour("Ali");`
`direBonjour("Sami");`

Déclaration d'une fonction

- Syntaxe générale:

```
function nomFonction() {  
  // instructions  
}
```

- Exemple fonction

Exemple du fonction

Fonctions sans paramètres

```
function afficherMessage() {  
  console.log("Bienvenue en JavaScript");  
}  
  
afficherMessage();
```

Fonctions avec paramètres

```
function somme(a, b) {  
  console.log(a + b);  
}  
  
somme(5, 3);
```

Exemple du fonction

Fonction qui retourne une valeur

```
function carre(x) {  
  return x * x;  
}  
  
let resultat = carre(4);  
console.log(resultat);
```

Fonctions anonymes (Fonction sans nom, stockée dans une variable)

```
let addition = function(a, b) {  
  return a + b;  
};  
  
console.log(addition(3, 4));
```

Exemple du fonction

- Fonctions fléchées

```
const somme = (a, b) => {  
  return a + b;  
};
```

- Version courte:

```
const somme = (a, b) => a + b;
```

Très utilisée en React

Les tableaux

Déclaration des tableaux

Un tableau est une structure qui permet de stocker plusieurs valeurs dans une seule variable.

- `let fruits = ["pomme", "banane", "orange"];`
- Les indices commencent à 0

Parcours de tableaux

- Avec une boucle for

```
for (let i = 0; i < notes.length; i++) {
  console.log(notes[i]);
}
```

- for...of (

```
for (let note of notes) {
  console.log(note);
}
```

Méthodes utiles

+Ajouter des éléments

```
notes.push(20); // fin
notes.unshift(8); // début
```

— Supprimer des éléments

```
notes.pop(); // fin
notes.shift(); // début
```

? Rechercher

```
console.log(notes.includes(15)); // true
```

map (transformation)

```
let notesPlus1 = notes.map(note => note + 1);
```

Méthodes utiles

filter (filtrer)

```
let admis = notes.filter(note => note >= 10);
```

Objets : propriétés et méthodes

- Un **objet** permet de regrouper des données liées.

```
let etudiant = {  
  nom: "Ali",  
  age: 20,  
  moyenne: 14.5  
};
```

Accès aux données

```
console.log(etudiant.nom);  
console.log(etudiant["age"]);
```

Différence tableau vs objet

- **Tableau** : indexé numériquement.
- **Objet** : indexé par des clés.

Objets : propriétés et méthodes

Méthodes dans un objet

```
let personne = {  
  nom: "Sami",  
  saluer: function() {  
    console.log("Bonjour " + this.nom);  
  }  
};  
  
personne.saluer();
```

Tableau d'objets

```
let etudiants = [  
  { nom: "Ali", note: 14 },  
  { nom: "Sara", note: 9 },  
  { nom: "Sami", note: 17 }  
];
```

Parcourir:

```
for (let e of etudiants) {  
  console.log(e.nom + " : " + e.note);  
}
```